

SQL/OLAP Tutorial

The objective of this tutorial is to get familiarized with the extensions provided by SQL in order to perform OLAP-like operations.

We will be using an online tool that provide an sql terminal for creating and querying relational databases: <https://www.db-fiddle.com/>

At the top left, you have a menu that allows you to choose a relational database system. In our case, we will use the most recent version of Potsgres. You have two text boxes, one is used for the creation and population of the database and the second is used to issue an SQL query. Note that this online tool does not save your database.

Setup a sample table

Let's get started by creating a new table called **sales** for the demonstration.

```
DROP TABLE IF EXISTS sales;
CREATE TABLE sales (
  brand VARCHAR NOT NULL,
  segment VARCHAR NOT NULL,
  quantity INT NOT NULL,
  PRIMARY KEY (brand, segment)
);
INSERT INTO sales (brand, segment, quantity)
VALUES
  ('ABC', 'Premium', 100),
  ('ABC', 'Basic', 200),
  ('XYZ', 'Premium', 100),
  ('XYZ', 'Basic', 300);
```

1. Grouping Set

A grouping set is a set of columns by which you group by using the GROUP BY clause. A grouping set is denoted by a comma-separated list of columns placed inside parentheses:

```
(column1, column2, ...)
```

For example, the following query uses the GROUP BY clause to return the number of products sold by brand and segment. In other words, it defines a grouping set of the brand and segment which is denoted by (brand, segment)

```
SELECT
  brand,
  segment,
  SUM (quantity)
FROM
```

```
sales
GROUP BY
brand,
segment;
```

	brand character varying	segment character varying	sum bigint
1	XYZ	Basic	300
2	ABC	Premium	100
3	ABC	Basic	200
4	XYZ	Premium	100

The following query finds the number of products sold by a brand. It defines a grouping set (brand):

```
SELECT
brand,
SUM (quantity)
FROM
sales
GROUP BY
brand;
```

	brand character varying	sum bigint
1	ABC	300
2	XYZ	400

The following query finds the number of products sold by segment. It defines a grouping set (segment):

```
SELECT
segment,
SUM (quantity)
FROM
sales
GROUP BY
segment;
```

	segment character varying	sum bigint
1	Basic	500
2	Premium	200

The following query finds the number of products sold for all brands and segments. It defines an empty grouping set which is denoted by ().

```
SELECT SUM (quantity) FROM sales;
```

	sum bigint
1	700

Suppose that you want all grouping sets by using a single query. To achieve this, you may use the UNION ALL to combine all the queries above.

Because UNION ALL requires all result sets to have the same number of columns with compatible data types, you need to adjust the queries by adding NULL to the selection list of each as shown below:

```
SELECT
  brand,
  segment,
  SUM (quantity)
FROM
  sales
GROUP BY
  brand,
  segment
UNION ALL
SELECT
  brand,
  NULL,
  SUM (quantity)
FROM
  sales
GROUP BY
  brand
UNION ALL
SELECT
  NULL,
  segment,
  SUM (quantity)
FROM
  sales
GROUP BY
  segment
UNION ALL
SELECT
  NULL,
  NULL,
  SUM (quantity)
FROM
  sales;
```

	brand character varying	segment character varying	sum bigint
1	XYZ	Basic	300
2	ABC	Premium	100
3	ABC	Basic	200
4	XYZ	Premium	100
5	ABC	[null]	300
6	XYZ	[null]	400
7	[null]	Basic	500
8	[null]	Premium	200
9	[null]	[null]	700

This query generated a single result set with the aggregates for all grouping sets.

Even though the above query works as you expected, it has two main problems.

- First, it is quite lengthy.
- Second, it has a performance issue because PostgreSQL has to scan the sales table separately for each query.

To make it more efficient, PostgreSQL provides the GROUPING SETS clause which is the subclause of the GROUP BY clause.

The GROUPING SETS allows you to define multiple grouping sets in the same query.

The general syntax of the GROUPING SETS is as follows:

```
SELECT
  c1,
  c2,
  aggregate_function(c3)
FROM
  table_name
GROUP BY
  GROUPING SETS (
    (c1, c2),
    (c1),
    (c2),
    ()
  );
```

In this syntax, we have four grouping sets (c1,c2), (c1), (c2), and ().

To apply this syntax to the above example, you can use GROUPING SETS clause instead of the UNION ALL clause like this:

```

SELECT
  brand,
  segment,
  SUM (quantity)
FROM
  sales
GROUP BY
  GROUPING SETS (
    (brand, segment), (brand), (segment), ()
  );

```

	brand character varying	segment character varying	sum bigint
1	[null]	[null]	700
2	XYZ	Basic	300
3	ABC	Premium	100
4	ABC	Basic	200
5	XYZ	Premium	100
6	ABC	[null]	300
7	XYZ	[null]	400
8	[null]	Basic	500
9	[null]	Premium	200

This query is much shorter and more readable. In addition, PostgreSQL will optimize the number of times it scans the sales table and will not scan multiple times.

Grouping function

The GROUPING() function accepts an argument which can be a column name or an expression:

```
GROUPING( column_name | expression)
```

The column_name or expression must match with the one specified in the GROUP BY clause.

The GROUPING() function returns bit 0 if the argument is a member of the current grouping set and 1 otherwise.

See the following example:

```

SELECT
  GROUPING(brand) grouping_brand,
  GROUPING(segment) grouping_segment,
  brand,
  segment,
  SUM (quantity)

```

```
FROM sales
GROUP BY
    GROUPING SETS ( (brand), (segment), () )
ORDER BY brand, segment;
```

	grouping_brand integer	grouping_segment integer	brand character varying	segment character varying	sum bigint
1	0	1	ABC	[null]	300
2	0	1	XYZ	[null]	400
3	1	0	[null]	Basic	500
4	1	0	[null]	Premium	200
5	1	1	[null]	[null]	700

As shown in the screenshot, when the value in the grouping_brand is 0, the sum column shows the subtotal of the brand.

When the value in the grouping_segment is zero, the sum column shows the subtotal of the segment.

You can use the GROUPING() function in the HAVING clause to find the subtotal of each brand like this:

```
SELECT
    GROUPING(brand) grouping_brand,
    GROUPING(segment) grouping_segment,
    brand,
    segment,
    SUM (quantity)
FROM sales
GROUP BY
    GROUPING SETS ( (brand), (segment), () )
HAVING GROUPING(brand) = 0
ORDER BY brand, segment;
```

	grouping_brand integer	grouping_segment integer	brand character varying	segment character varying	sum bigint
1	0	1	ABC	[null]	300
2	0	1	XYZ	[null]	400

2. Roll Up

The PostgreSQL ROLLUP is a subclause of the GROUP BY clause that offers a shorthand for defining multiple grouping sets.

ROLLUP does not generate all possible grouping sets based on the specified columns. It just makes a subset of those.

The ROLLUP assumes a hierarchy among the input columns and generates all grouping sets that make sense considering the hierarchy. This is the reason why ROLLUP is often used to generate the subtotals and the grand total for reports.

For example, the CUBE (c1,c2,c3) makes all eight possible grouping sets:

```
(c1, c2, c3)
(c1, c2)
(c2, c3)
(c1, c3)
(c1)
(c2)
(c3)
()
```

However, the ROLLUP(c1,c2,c3) generates only four grouping sets, assuming the hierarchy $c1 > c2 > c3$ as follows:

```
(c1, c2, c3)
(c1, c2)
(c1)
()
```

A common use of ROLLUP is to calculate the aggregations of data by year, month, and date, considering the hierarchy $\text{year} > \text{month} > \text{date}$

The following illustrates the syntax of the PostgreSQL ROLLUP:

```
SELECT c1, c2, c3, aggregate(c4)
FROM
  table_name
GROUP BY
  ROLLUP (c1, c2, c3);
```

It is also possible to do a partial roll up to reduce the number of subtotals generated.

```
SELECT c1, c2, c3, aggregate(c4)
FROM
  table_name
GROUP BY
  c1, ROLLUP (c2, c3);
```

PostgreSQL ROLLUP examples

If you haven't create the sales table, you can use the following script:

```
DROP TABLE IF EXISTS sales;
CREATE TABLE sales (
  brand VARCHAR NOT NULL,
  segment VARCHAR NOT NULL,
  quantity INT NOT NULL,
  PRIMARY KEY (brand, segment)
);
INSERT INTO sales (brand, segment, quantity)
VALUES
  ('ABC', 'Premium', 100),
  ('ABC', 'Basic', 200),
  ('XYZ', 'Premium', 100),
  ('XYZ', 'Basic', 300);
```

The following query uses the ROLLUP clause to find the number of products sold by brand (subtotal) and by all brands and segments (total).

```
SELECT
  brand,
  segment,
  SUM(quantity)
FROM
  sales
GROUP BY
  ROLLUP (brand, segment)
ORDER BY brand, segment;
```

brand	segment	sum
ABC	Basic	200
ABC	Premium	100
ABC	(Null)	300
XYZ	Basic	300
XYZ	Premium	100
XYZ	(Null)	400
(Null)	(Null)	700

As you can see clearly from the output, the third row shows the sales of the ABC brand, the sixth row displays sales of the the XYZ brand. The last row shows the grand total for all brands and segments. In this example, the hierarchy is brand > segment.

If you change the order of brand and segment, the result will be different as follows:

```
SELECT
  segment,
  brand,
  SUM(quantity)
```



```
FROM  
  sales  
GROUP BY  
  ROLLUP (segment, brand)  
ORDER BY segment, brand;
```

segment	brand	sum
Basic	ABC	200
Basic	XYZ	300
Basic	(Null)	500
Premium	ABC	100
Premium	XYZ	100
Premium	(Null)	200
(Null)	(Null)	700

In this case, the hierarchy is the segment > brand.

3. Cube

CUBE is a subclause of the GROUP BY clause. The CUBE allows you to generate multiple grouping sets. The following illustrates the syntax of the CUBE subclause:

```
SELECT
  c1,
  c2,
  c3,
  aggregate (c4)
FROM
  table_name
GROUP BY
  CUBE (c1, c2, c3);
```

In this syntax:

- First, specify the CUBE subclause in the the **GROUP BY** clause of the **SELECT** statement.
- Second, in the select list, specify the columns (dimensions or dimension columns) which you want to analyze and **aggregation function** expressions.
- Third, in the GROUP BY clause, specify the dimension columns within the parentheses of the CUBE subclause.

The query generates all possible grouping sets based on the dimension columns specified in CUBE. The CUBE subclause is a short way to define multiple grouping sets so the following are equivalent:

```
CUBE(c1,c2,c3)
```

```
GROUPING SETS (
  (c1,c2,c3),
  (c1,c2),
  (c1,c3),
  (c2,c3),
  (c1),
  (c2),
  (c3),
  ()
)
```

In general, if the number of columns specified in the CUBE is n , then you will have 2^n combinations.

PostgreSQL allows you to perform a partial cube to reduce the number of aggregates calculated. The following shows the syntax:

```
SELECT
  c1,
  c2,
```

```

c3,
aggregate (c4)
FROM
table_name
GROUP BY
c1,
CUBE (c1, c2);

```

PostgreSQL CUBE examples

We will use the sales table created in the GROUPING SETS tutorial for the demonstration.

brand	segment	quantity
ABC	Premium	100
ABC	Basic	200
XYZ	Premium	100
XYZ	Basic	300

The following query uses the CUBE subclause to generate multiple grouping sets:

```

SELECT
brand,
segment,
SUM (quantity)
FROM
sales
GROUP BY
CUBE (brand, segment)
ORDER BY brand, segment;

```

Here is the output:

brand	segment	sum
ABC	Basic	200
ABC	Premium	100
ABC	(Null)	300
XYZ	Basic	300
XYZ	Premium	100
XYZ	(Null)	400
(Null)	Basic	500
(Null)	Premium	200
(Null)	(Null)	700

The following query performs a partial cube:

```

SELECT
brand,
segment,
SUM (quantity)
FROM
sales
GROUP BY

```

```
brand,  
CUBE (segment)  
ORDER BY brand, segment;
```

brand	segment	sum
▶ ABC	Basic	200
ABC	Premium	100
ABC	(Null)	300
XYZ	Basic	300
XYZ	Premium	100
XYZ	(Null)	400